

Análisis y diseño con patrones

PEC2: Patrones de diseño y de asignación de responsabilidades

Índice

Pregunta 1 (20%)	3
Pregunta 1.1 (5 %).....	3
Pregunta 1.2 (15 %).....	4
Pregunta 2 (20 %)	5
Pregunta 2.1 (5 %).....	5
Pregunta 2.2 (15 %).....	7
Pregunta 3 (30 %)	8
Pregunta 3.1. (5 %).....	8
Pregunta 3.2 (12,5 %).....	10
Pregunta 4 (30 %)	12
Pregunta 4.1. (2.5 %).....	12
Pregunta 4.2. (12,5 %).....	13
Pregunta 4.3. (15 %).....	14

Compromiso de autorresponsabilidad

Certifico que he realizado la PEC2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de 0 %.

Pregunta 1 (20%)

En una red de computación neuronal distribuida, existe un componente crítico denominado *GlobalStateCoordinator*. Por razones de coherencia sináptica y para evitar conflictos de memoria, el sistema requiere que exista un único punto de acceso y control para la gestión de la configuración de red. Si dos coordinadores intentan gestionar la red simultáneamente, la estructura de datos se corrompería. Por tanto, el sistema debe garantizar que cualquier módulo que necesite consultar o modificar la configuración lo haga a través de un mismo punto en todo el ciclo de vida de la ejecución.

Por otro lado, la red está compuesta por múltiples *ProcessingNode*. Estos nodos necesitan ajustar su frecuencia de cálculo inmediatamente cada vez que la configuración en el coordinador cambia (p. ej., ante un aumento de temperatura o carga de trabajo). Sin embargo, el sistema debe ser eficiente:

- Los nodos no deben preguntar constantemente al coordinador si hay cambios.
- El coordinador mantiene una lista de los nodos activos.
- Cada nodo está especializado en un "Tipo de Tarea" (p. ej., Vision, Audio). Cuando el coordinador emite una actualización, esta incluye una "Etiqueta de Tarea". El coordinador debe recorrer sus nodos registrados y notificar únicamente a aquellos cuya especialización coincida con la etiqueta de la actualización, evitando interrumpir el trabajo de los nodos no afectados.

Se pide:

Pregunta 1.1 (5 %)

¿Qué patrón o patrones usarías para modelar este problema?

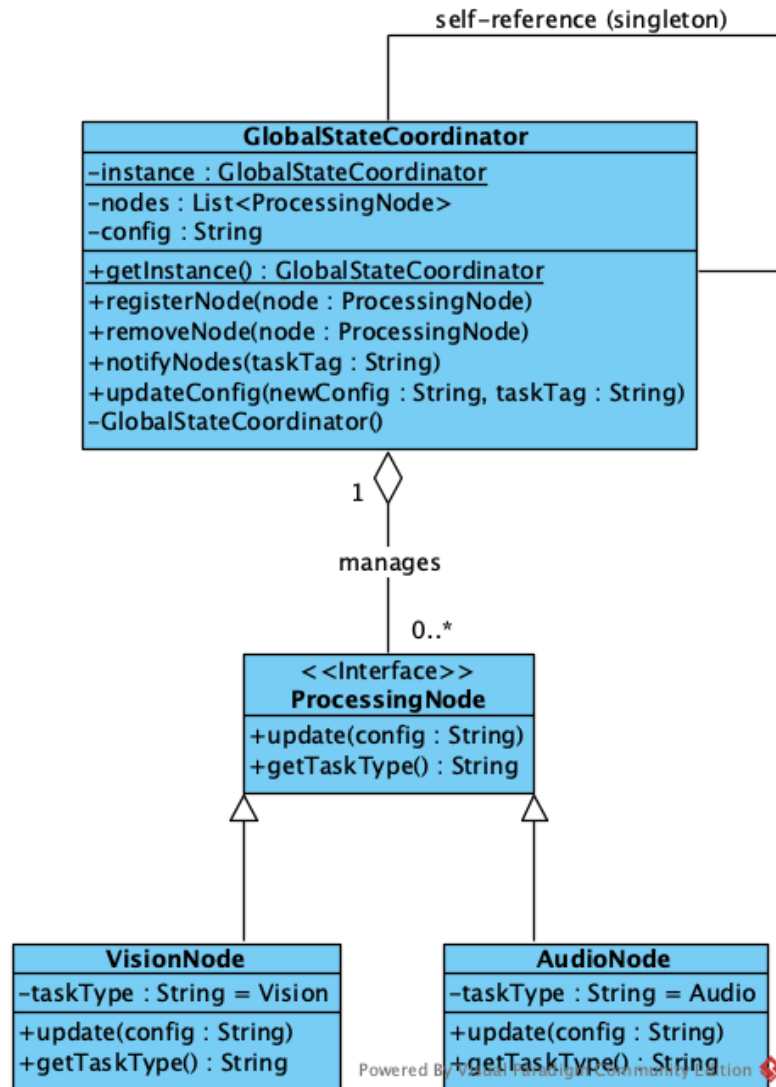
El enunciado especifica que "debe existir un único punto de acceso" para evitar la corrupción de la estructura de datos. Bajo esta premisa, el patrón ***singleton*** hace que solo exista una instancia del coordinador *GlobalStateCoordinator* en todo el sistema, lo cual proporciona un punto de acceso global único para cualquier módulo que necesite gestionar la configuración.

Además, el patrón ***observer*** es la clave para la comunicación eficiente entre el coordinador (*GlobalStateCoordinator*) y los nodos (*ProcessingNode*), puesto que permite que el coordinador notifique automáticamente a los nodos cuando ocurre un cambio. De esta forma, se cumple con el requisito que determina que los nodos no deben preguntar constantemente si no hay cambios.

Por último, como el sistema necesita que el coordinador recorra su lista de nodos registrados para enviar notificaciones selectivas, el patrón ***iterator*** nos ayuda a recorrer esta colección de nodos de manera secuencial sin exponer la estructura interna de la lista, lo que facilita la lógica de filtrado por "Tipo de Tarea".

Pregunta 1.2 (15 %)

Muestra el diagrama de clases UML que ilustra tu solución.



Pregunta 2 (20 %)

La corporación NeuralLink está desarrollando un dispositivo implantable que estabiliza la actividad cerebral en tiempo real. El sistema debe ser capaz de reaccionar de forma distinta según el nivel de alerta del usuario, pero manteniendo los algoritmos de procesamiento físico totalmente independientes de la lógica de control.

El dispositivo opera en tres modalidades distintas: *Safe*, *Active* e *Intensive*. Cada modalidad decide cómo actuar. No solo se limitan a indicar el modo, sino que aplican políticas de seguridad sobre los datos. Por ejemplo, el modo *Safe* limitará cualquier resultado por encima de un umbral para proteger al paciente, mientras que el modo *Intensive* aplicará multiplicadores de potencia ante una crisis. Además, cada modalidad es responsable de monitorizar la estabilidad: si el modo *Active* detecta que la señal es demasiado errática, él mismo debe ordenar al regulador pasar a modo *Intensive*.

Independientemente de la modalidad de operación, la operación técnica de estabilizar el impulso depende de un algoritmo de filtrado. Este algoritmo es una pieza de lógica pura que puede variar según el modelo del implante o la necesidad del paciente (p. ej., Algoritmo Lineal, Algoritmo Gaussiano, etc.). Se requiere que estos algoritmos sean intercambiables y que el sistema pueda usar uno u otro sin tener que modificar el código de las modalidades de funcionamiento. Así, una misma modalidad (p. ej., *Active*) podría realizar sus tareas usando un algoritmo lineal en un paciente y uno gaussiano en otro.

Se pide:

Pregunta 2.1 (5 %)

Identifica qué patrones permiten gestionar las casuísticas de este problema. En caso de que haya más de un patrón, explica cómo interactúan entre sí.

En primera instancia, identificamos la aplicación del patrón **state** para gestionar las modalidades *Safe*, *Active* e *Intensive*, puesto que el dispositivo debe cambiar su comportamiento interno dependiendo de su estado de alerta. Este patrón permite que el objeto regulador delegue la lógica de control a una clase específica que representa el modo actual.

En cuanto a la gestión de algoritmos de filtrado, el enunciado exige que los algoritmos sean intercambiables y que la modalidad pueda usarlos sin conocer su implementación interna. Por lo tanto, el patrón **strategy** es el que mejor se adapta a esta premisa, ya que nos ayuda a definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. Esto facilita que una modalidad mantenga una referencia a una interfaz de filtrado, permitiendo cambiar el "cómo" se estabiliza el impulso sin alterar el "cuándo" o el "por qué".

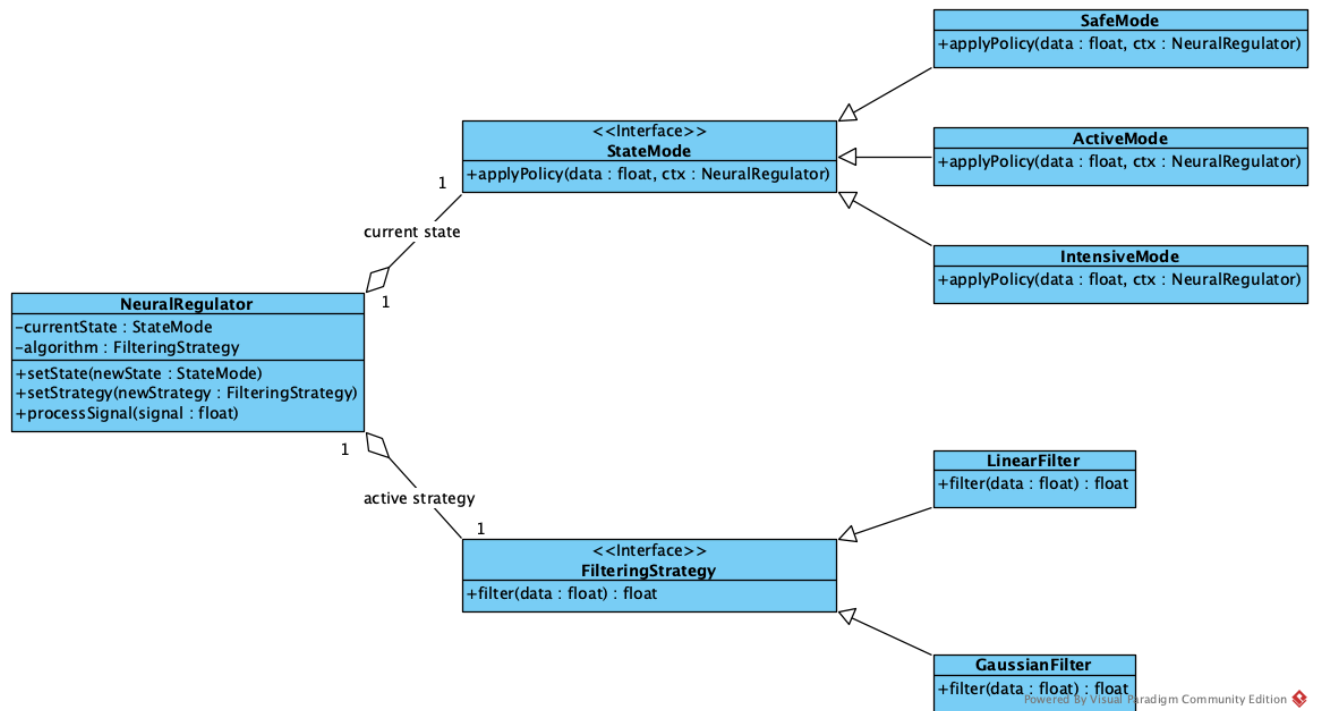
La colaboración entre ambos patrones crea un sistema desacoplado en el que el sistema principal mantiene una instancia del estado actual. Cuando el sistema recibe una señal cerebral, delega la

acción al objeto de uno de los estados (*Safe*, *Active* o *Intensive*), el cual utiliza una estrategia de filtrado. La modalidad (estado) decide qué reglas de negocio aplicar, pero delega el cálculo físico al algoritmo de filtrado (estrategia). Gracias a esta interacción, se pueden realizar cambios en dos ejes distintos sin que se afecten entre sí. Por ejemplo, podemos añadir una nueva modalidad sin tocar los algoritmos de filtrado o modificar una ya existente sin necesidad de modificar a las otras.

Aunque el patrón de *inyección de dependencias* podría usarse para introducir la estrategia en el estado, y un *factory method* podría crear los estados, los patrones fundamentales que definen la arquitectura y resuelven el problema de lógica y variabilidad planteado son, estrictamente, **state** y **strategy**.

Pregunta 2.2 (15 %)

Realiza el diagrama de clases que ilustra este problema.



Pregunta 3 (30 %)

La institución Archivo Digital utiliza un sistema para asegurar que los documentos escaneados no han sido alterados. El núcleo del sistema es un controlador llamado *DocumentValidator*, que se encarga de procesar las solicitudes de archivo. Para proteger al sistema de cambios tecnológicos, el controlador utiliza una interfaz propia llamada *IntegrityChecker* con un único método *isValid(Document d): Boolean*.

Recientemente, se ha adquirido una licencia de una herramienta forense avanzada llamada *UltraScannerSDK*. Sin embargo, esta librería externa es incompatible con nuestra interfaz. Para utilizarla, el flujo de trabajo es el siguiente:

1. Cargar el archivo con el método *initScan(File f)*.
2. Ejecutar el análisis mediante *performForensicAnalysis()*.
3. Extraer dos valores: *integrityScore* (un valor decimal entre 0 y 1) y *reliabilityIndex* (un porcentaje).

La dirección técnica indica que un documento solo es válido si el *integrityScore* es superior a 0.8 y el *reliabilityIndex* es mayor al 90%,

Una vez obtenida la respuesta del análisis, el controlador debe finalizar el proceso: si el documento es válido, se debe invocar el método *markAsValidated()* del documento para permitir su archivo definitivo; si el análisis falla, se debe invocar *markAsRejected()* para enviarlo a revisión manual.

Se pide:

Pregunta 3.1. (5 %)

Indica qué patrón o patrones de diseño deben aplicarse para integrar *UltraScannerSDK* en el sistema.

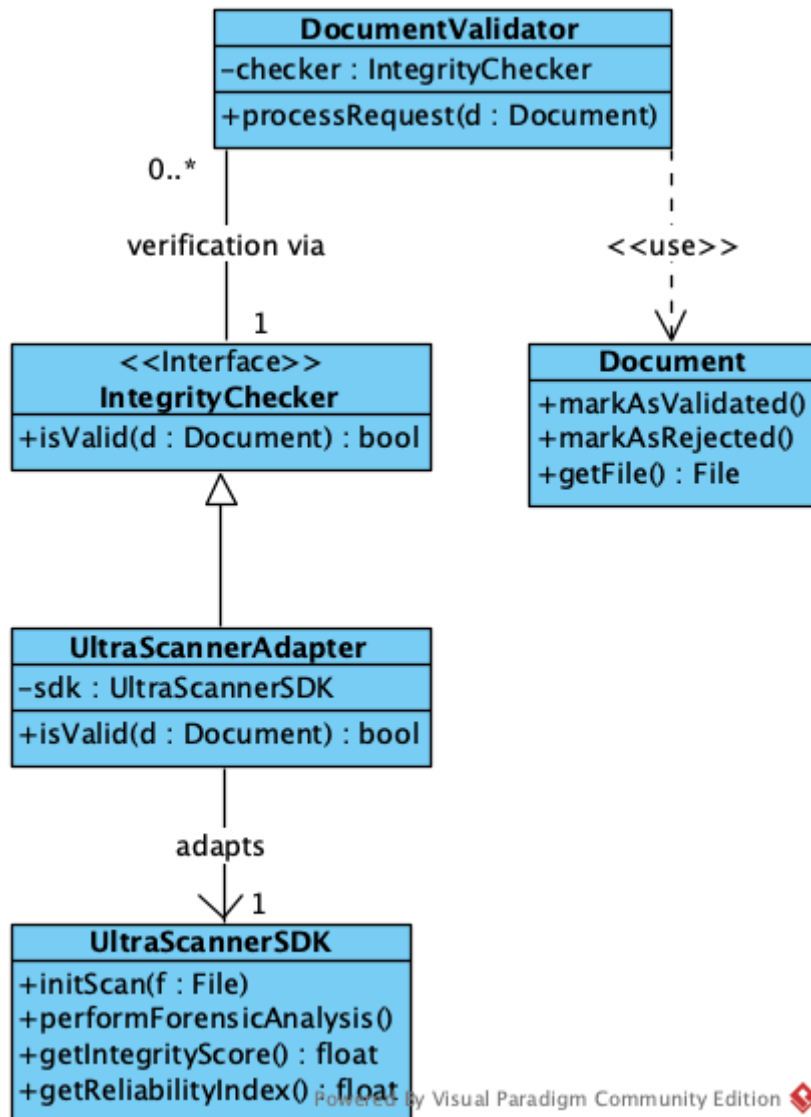
En el sistema planteado por el enunciado existe una incompatibilidad de interfaces, ya que el sistema espera un objeto que implemente *IntegrityChecker* con el método *isValid()*, mientras que la librería externa tiene un flujo de trabajo totalmente distinto al utilizar métodos como *initScan* o *performForensicAnalysis*. Para solventar este problema, podemos aplicar el patrón **adapter** creando una clase adaptadora (ej. *UltraScannerAdapter*) que implemente *IntegrityChecker*. Dentro del método *isValid()*, el adaptador traducirá la llamada realizando internamente los tres pasos que requiere el SDK y evaluando la lógica de negocio (*score > 0.8* y *reliability > 90%*).

Al aplicar el patrón *adapter* para resolver la comunicación técnica, también estamos, a su vez, aplicando el patrón **expert**, ya que el *adapter* se convierte en el experto en la interpretación de los datos crudos del SDK para devolver un booleano.

Por último, para que el controlador *DocumentValidator* pueda utilizar el nuevo *UltraScannerSDK* sin estar acoplado a él, se le debe inyectar el adaptador a través de la interfaz *IntegrityChecker*, por lo que usaremos la **inyección de dependencias** para ello con tal de proteger al sistema de cambios tecnológicos. Esto nos permitirá en el futuro cambiar las herramientas forenses por otras sin necesidad de modificar el código del controlador.

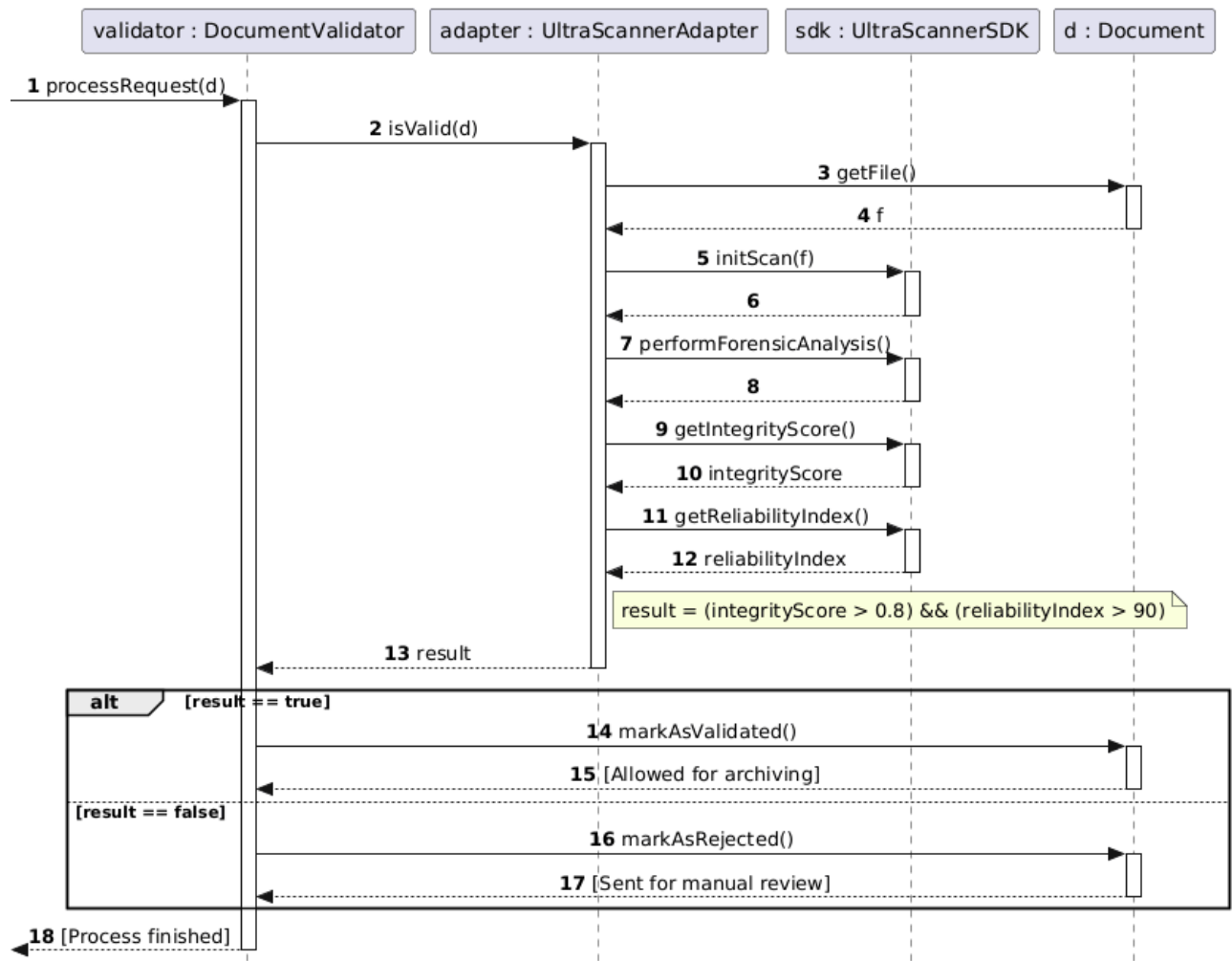
Pregunta 3.2 (12,5 %)

Muestra el diagrama de clases UML con la integración de la nueva librería



Pregunta 3.3 (15 %)

Realiza el diagrama de secuencia de la operación de validación utilizando la herramienta externa.



Pregunta 4 (30 %)

La constructora Percón necesita un software para generar presupuestos automáticos de proyectos, no se fía de utilizar ninguna IA. El coste final de un proyecto (*Project*) se rige por una fórmula fija: ***totalBudget = (executionCost + materialCost) * (1 + profitMargin)***. Para realizar este cálculo es necesario tener en cuenta los dos tipos de proyectos principales que existen: Edificación (*HousingProject*) y Obra Civil (*CivilEngineeringProject*).

- En la Edificación, el *executionCost* se calcula multiplicando los metros cuadrados por una tasa fija de vivienda.
- En la Obra Civil, el *executionCost* se calcula mediante un canon gubernamental por kilómetro de superficie.

Por otro lado, el proyecto siempre requiere un Lote de Materiales (*MaterialBatch*). El cálculo del coste de los materiales también sigue un proceso fijo: ***totalMaterialCost = marketPrice + logisticsSurcharge***.

- Si el lote es Standard, *logisticsSurcharge* es siempre cero.
- Si el lote es Special (Materiales Peligrosos o Pesados), el *logisticsSurcharge* incluye una tasa de transporte especial y un seguro de riesgo.

Se pide:

Pregunta 4.1. (2.5 %)

Identifica qué patrón es adecuado para garantizar que el algoritmo de cálculo del presupuesto no sea modificado por las subclases, pero permitiendo que estas definan los costes específicos. Menciona en qué parte se aplica en concreto.

El patrón ***template method*** es ideal cuando se tiene un algoritmo con una estructura fija donde algunos de esos pasos deben ser implementados o personalizados por las subclases sin cambiar la estructura global.

En el caso del sistema planteado, el patrón se aplica en la clase base *Project*, el cual define un método “final” (por ejemplo, *calculateTotalBudget()*) que contiene la fórmula fija para obtener el *totalBudget*. Al declararlo como “final” nos aseguramos de que la estructura del cálculo no sea modificada.

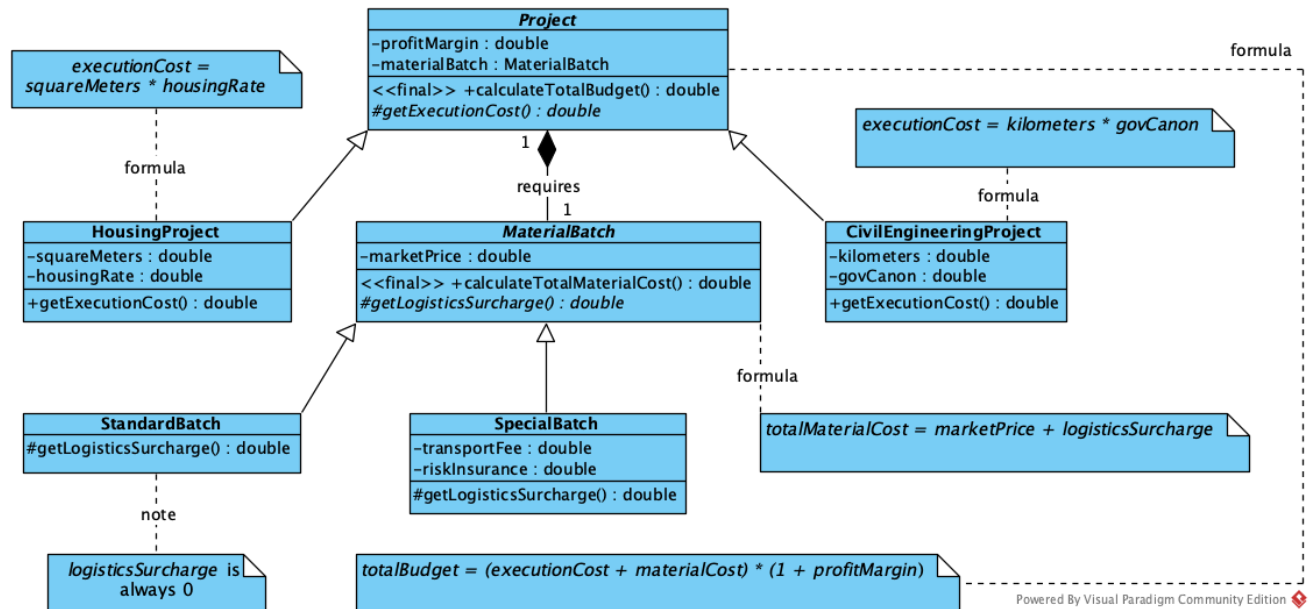
Dentro de ese método fijo, se hace una llamada a un método abstracto llamado *getExecutionCost()*. La clase *HousingProject* implementa *getExecutionCost()* realizando el cálculo de metros cuadrados por tasa fija. La clase *CivilEngineeringProject* implementa *getExecutionCost()* aplicando el canon gubernamental por kilómetro.

De esta forma, la jerarquía de proyectos hace que la lógica de negocio de Percón permanezca intacta y centralizada, mientras delegamos la variabilidad del coste de ejecución a cada tipo de obra específico.

Es importante tener en cuenta que este mismo patrón puede aplicarse de forma análoga en la jerarquía de *MaterialBatch* para el cálculo del *totalMaterialCost*, donde el *template method* define la suma del precio de mercado y el recargo, y las subclases (*Standard* y *Special*) definen cuánto vale dicho recargo.

Pregunta 4.2. (12,5 %)

Muestra el diagrama de clases UML completo. Es obligatorio que el diagrama refleje qué métodos son los que definen el esqueleto del algoritmo.



Pregunta 4.3. (15 %)

Diagrama de secuencia de la operación *Project::calculateTotalBudget()*: *Float* cuando se solicita para un proyecto de Edificación que utiliza un lote de Materiales Especiales.

